

```

# STIPENDI NOIPA =====

# 1) PACCHETTI ----
# install.packages(c("tibble", "dplyr", "readr", "purrr", "openxlsx"))

library(tibble)
library(dplyr)
library(readr)
library(purrr)
library(openxlsx)

# 2) AVVERTENZE METODOLOGICHE ----
# Questo script usa una logica di costruzione degli URL ricavata
# empiricamente dagli URL generati dal portale NoiPA.
#
# La procedura dipende dalla struttura corrente del portale:
# - portlet id
# - parametri della query string
# - formato ZIP contenente CSV
#
# Se il portale cambia struttura, la pipeline potrebbe richiedere
# aggiornamenti.
#
# Nota sulla periodicità:
# quasi tutti i dataset trattati sono mensili.
# Il dataset "EntryCertificazioniUniche" è trattato come annuale nel
# reporting,
# anche se il download sul portale passa tramite il valore tecnico mese =
# 09.

# 3) CATALOGO DATASET ----
catalogo_noipa <- tibble::tibble(
  dataset_name = c(
    "Amministrati per provincia di residenza",
    "Amministrati",
    "Modalità di accredito degli stipendi",
    "Mobilità degli Amministrati",
    "Modalità di accesso",
    "Struttura organizzativa Amministrazioni",
    "Inquadramento degli amministratori",
    "Inquadramenti contrattuali per Amministrazione",
    "Motivi assunzione",
    "Motivi di cessazione",
    "Evoluzione delle detrazioni per familiari a carico",
    "Assegni al nucleo familiare",
    "Assenze contabilizzate a fini economici",
    "Ritenute previdenziali",
    "Ritenute fiscali",
    "Ritenute per Ricorso al Credito",
    "Ritenute per Adesioni sindacali",
    "Redditi di lavoro dipendente e assimilati certificati agli
Amministrati",
    "Amministrati per Fascia di Reddito"
  ),
  dataset_id = c(
    "EntryResidenti",

```

```

    "EntryAmministrati",
    "EntryAccreditoStipendi",
    "EntryPendolarismo",
    "EntryAccessoAmministrati",
    "EntryStrutturaOrganizzativa",
    "EntryInquadramenti",
    "EntryContrattiGestiti",
    "EntryMotivoAssunzione",
    "EntryMotivoCessazione",
    "EntryDetrazioniFamiliari",
    "EntryAssegniFamiliari",
    "EntryAssenzeContabilizzate",
    "EntryCedolinoRitenutePrevidenziali",
    "EntryCedolinoRitenuteFiscali",
    "EntryRitenutePrestiti",
    "EntryRitenuteSindacali",
    "EntryCertificazioniUniche",
    "EntryAmministratiPerFasciaDiReddito"
  ),
  anno = c(
    2025, 2025, 2025, 2025, 2025,
    2025, 2025, 2025, 2025, 2025,
    2025, 2022, 2025, 2025, 2025,
    2025, 2025, 2025, 2025
  ),
  mese = c(
    12, 12, 12, 12, 12,
    12, 12, 12, 12, 12,
    12, 12, 12, 12, 12,
    12, 12, 9, 12
  ),
  periodicit  = c(
    rep("mensile", 17),
    "annuale",
    "mensile"
  )
)

# 4) PARAMETRI BASE DEL PORTALE ----
noipa_base_url <- "https://dati-noipa.mef.gov.it/cl/web/open-
data/dataset"
noipa_portlet_id <-
"it_gov_mef_opendata_portlet_NoipaOpendataPortlet_INSTANCE_k0QJbYynlaqN"

# 5) FUNZIONI DI SUPPORTO ----
# Costruisce l'URL di download del file CSV (in realt  ZIP contenente
CSV)
build_noipa_csv_url <- function(dataset_id, anno, mese = NA) {
  url <- paste0(
    noipa_base_url,
    "?p_p_id=", noipa_portlet_id,
    "&p_p_lifecycle=2",
    "&p_p_state=normal",
    "&p_p_mode=view",
    "&p_p_cacheability=cacheLevelPage",
    "&_", noipa_portlet_id, "_anno=", anno,

```

```

    "&_", noipa_portlet_id, "_formato=CSV"
  )

  if (!is.na(mese)) {
    url <- paste0(
      url,
      "&_", noipa_portlet_id, "_mese=", sprintf("%02d", mese)
    )
  }

  url <- paste0(
    url,
    "&_", noipa_portlet_id, "_id=", dataset_id,
    "&_", noipa_portlet_id, "_id=", dataset_id,
    "&_", noipa_portlet_id,
    "_jspPage=%2Fdettaglio%2FdettaglioDataSet.jsp",
    "&p_p_lifecycle=1",
    "&_", noipa_portlet_id, "_javax.portlet.action=getDettaglio"
  )

  url
}

# Costruisce l'URL della pagina di dettaglio del dataset
build_noipa_detail_url <- function(dataset_id) {
  paste0(
    noipa_base_url,
    "?p_p_id=", noipa_portlet_id,
    "&p_p_lifecycle=1",
    "&p_p_state=normal",
    "&p_p_mode=view",
    "&_", noipa_portlet_id, "_javax.portlet.action=getDettaglio",
    "&_", noipa_portlet_id, "_id=", dataset_id
  )
}

# Scarica il file ZIP, estrae il CSV e lo legge in R
read_noipa_dataset <- function(dataset_id, anno, mese, delim = ",") {
  csv_url <- build_noipa_csv_url(
    dataset_id = dataset_id,
    anno = anno,
    mese = mese
  )

  tmp_file <- tempfile(fileext = ".zip")

  utils::download.file(
    url = csv_url,
    destfile = tmp_file,
    mode = "wb",
    method = "libcurl"
  )

  tmp_dir <- tempfile(pattern = "noipa_unzip_")
  dir.create(tmp_dir)

  utils::unzip(
    zipfile = tmp_file,

```

```

    exdir = tmp_dir
  )

  csv_files <- list.files(
    path = tmp_dir,
    pattern = "\\*.csv$",
    full.names = TRUE,
    recursive = TRUE
  )

  if (length(csv_files) == 0) {
    stop("Nessun file CSV trovato nello ZIP scaricato.")
  }

  csv_file <- csv_files[1]

  df <- readr::read_delim(
    file = csv_file,
    delim = delim,
    show_col_types = FALSE
  )

  list(
    data = tibble::as_tibble(df),
    csv_url = csv_url,
    csv_file = csv_file
  )
}

# 6) ESTRAZIONE DATASET E COSTRUZIONE OUTPUT ----
estrazioni_noipa <- purrr::pmap(
  catalogo_noipa,
  function(dataset_name, dataset_id, anno, mese, periodicità) {

    out <- tryCatch(
      read_noipa_dataset(
        dataset_id = dataset_id,
        anno = anno,
        mese = mese,
        delim = ",",
      ),
      error = function(e) NULL
    )

    periodo_disponibile <- if (periodicità == "annuale") {
      as.character(anno)
    } else {
      paste0(sprintf("%02d", mese), "/", anno)
    }

    if (is.null(out)) {
      metadata_row <- tibble::tibble(
        `Dati stipendi NoiPA - dataset` = dataset_name,
        `periodo/annualità disponibili` = NA_character_,
        `ultimo aggiornamento disponibile` = periodo_disponibile,
        `variabili di interesse` = NA_character_,
        `n_osservazioni` = NA_integer_,
      )
    }
  }
)

```

```

      `n_variabili` = NA_integer_,
      `modalità di accesso` = "Download file ZIP contenente CSV",
      `limiti tecnici (rate limit)` = NA_character_,
      `formati scarico dati` = "ZIP/CSV",
      `note` = paste0("dataset_id: ", dataset_id, "; errore lettura
dataset")
    )

    return(list(
      metadata = metadata_row,
      variables = NULL
    ))
  }

df <- out$data

metadata_row <- tibble::tibble(
  `Dati stipendi NoiPA - dataset` = dataset_name,
  `periodo/annualità disponibili` = NA_character_,
  `ultimo aggiornamento disponibile` = periodo_disponibile,
  `variabili di interesse` = paste(names(df), collapse = " | "),
  `n_osservazioni` = nrow(df),
  `n_variabili` = ncol(df),
  `modalità di accesso` = "Download file ZIP contenente CSV",
  `limiti tecnici (rate limit)` = NA_character_,
  `formati scarico dati` = "ZIP/CSV",
  `note` = paste0("dataset_id: ", dataset_id)
)

variables_table <- tibble::tibble(
  dataset_name = dataset_name,
  nome_file = basename(out$csv_file),
  variabile = names(df),
  posizione_variabile = seq_along(names(df))
)

list(
  metadata = metadata_row,
  variables = variables_table
)
}
)

mappatura_noipa <- purrr::map_dfr(estrazioni_noipa, "metadata")
variabili_noipa <- purrr::map_dfr(estrazioni_noipa, "variables")

# 7) EXPORT EXCEL ----
wb <- openxlsx::createWorkbook()

header_style <- openxlsx::createStyle(
  fontColour = "white",
  fgFill = "#2E75B6",
  halign = "center",
  textDecoration = "bold",
  border = "Bottom"
)

```

```

openxlsx::addWorksheet(wb, "metadata")
openxlsx::writeData(wb, "metadata", mappatura_noipa, withFilter = TRUE)
openxlsx::addStyle(
  wb,
  sheet = "metadata",
  style = header_style,
  rows = 1,
  cols = 1:ncol(mappatura_noipa),
  gridExpand = TRUE
)
openxlsx::freezePane(wb, "metadata", firstRow = TRUE)
openxlsx::setColWidths(wb, "metadata", cols = 1:ncol(mappatura_noipa),
widths = "auto")

openxlsx::addWorksheet(wb, "variabili")
openxlsx::writeData(wb, "variabili", variabili_noipa, withFilter = TRUE)
openxlsx::addStyle(
  wb,
  sheet = "variabili",
  style = header_style,
  rows = 1,
  cols = 1:ncol(variabili_noipa),
  gridExpand = TRUE
)
openxlsx::freezePane(wb, "variabili", firstRow = TRUE)
openxlsx::setColWidths(wb, "variabili", cols = 1:ncol(variabili_noipa),
widths = "auto")

openxlsx::saveWorkbook(
  wb,
  file = "mappatura_stipendi_noipa.xlsx",
  overwrite = TRUE
)

```